

Codemonk Backend API Project

1. API Documentation

Backend API Project 1.0.0 OAS 3.0

</api/schema/>

API documentation for Codemonk Backend Project

[Authorize](#)

paragraphs

- GET /api/paragraphs/
- POST /api/paragraphs/input/
- GET /api/paragraphs/search/

users

- POST /api/users/login/
- POST /api/users/register/
- POST /api/users/token/refresh/

2. Paragraph Module

GET
</api/paragraphs/>

paragraphs

GET /api/paragraphs/

Parameters

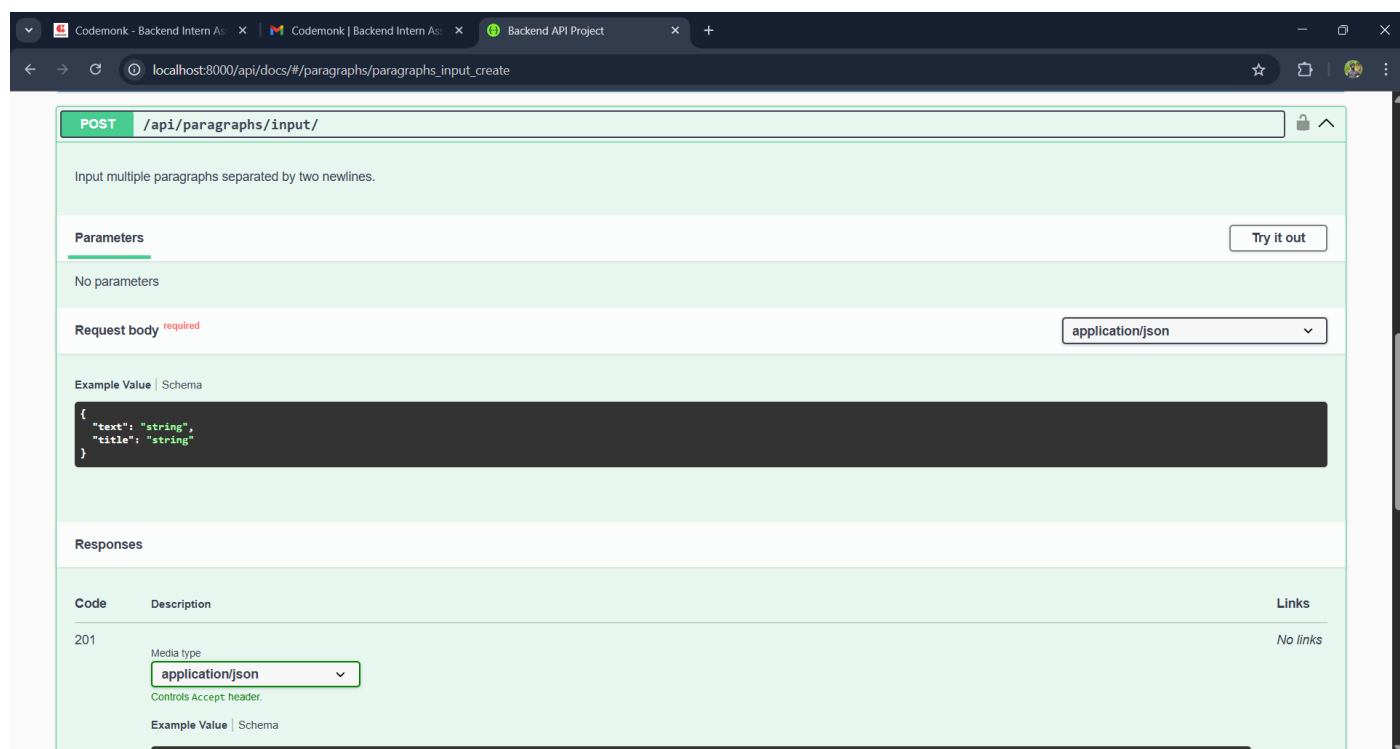
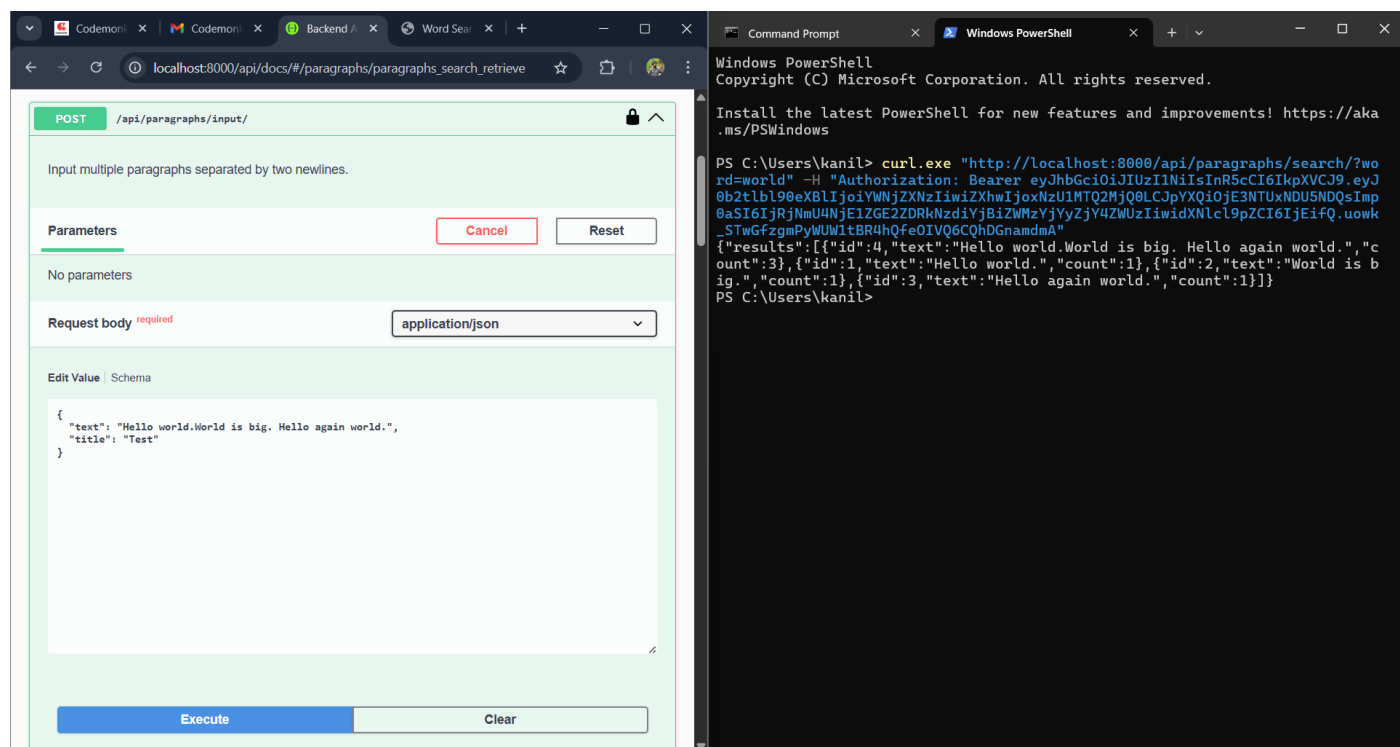
No parameters

Responses

Code	Description	Links
200	Media type application/json Controls Accept header. Example Value Schema <pre>[{"id": 0, "text": "string", "created_at": "2025-08-13T16:59:12.878Z"}]</pre>	No links

POST /api/paragraphs/input/

[/api/paragraphs/input/](#)

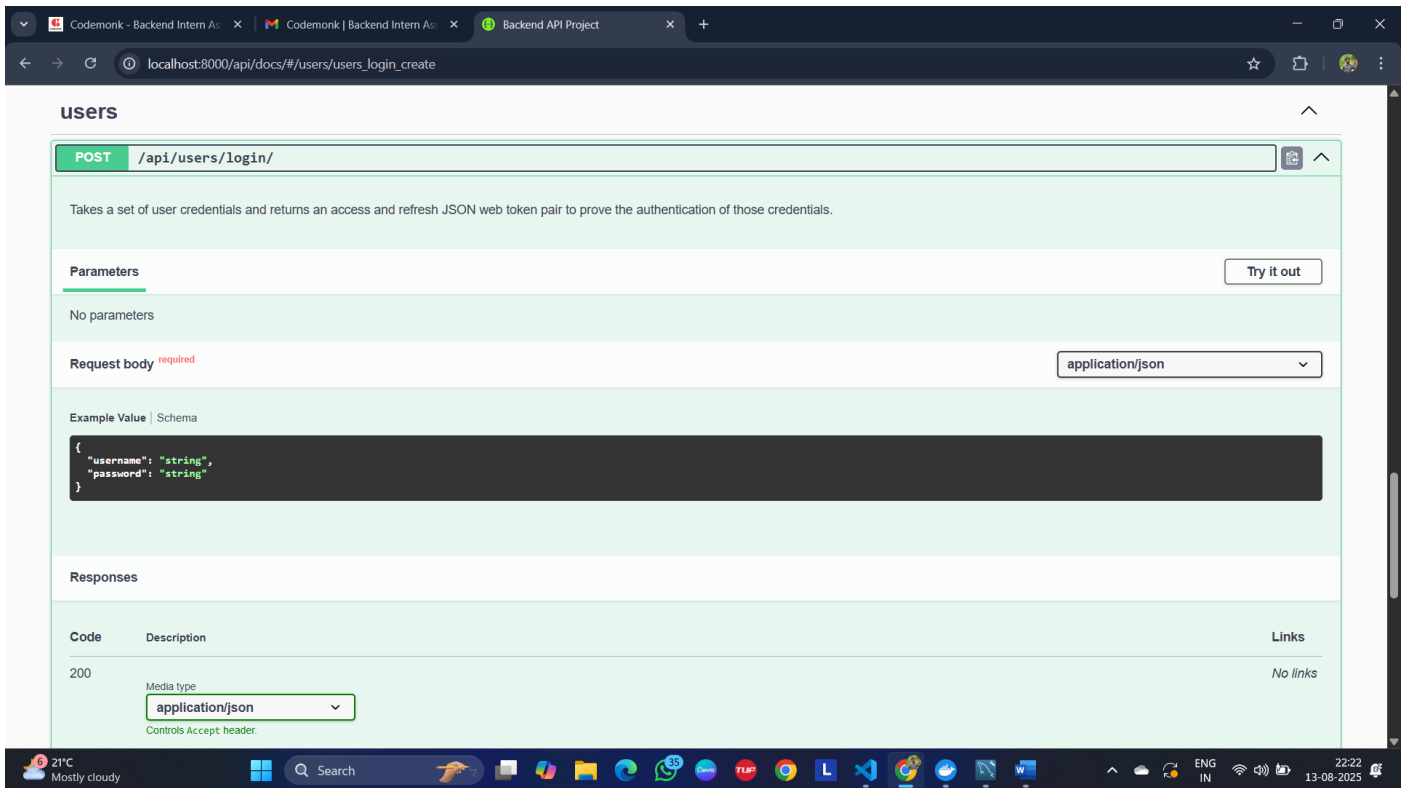
</api/paragraphs/search/>

3. User Module

I. Login into app

POST

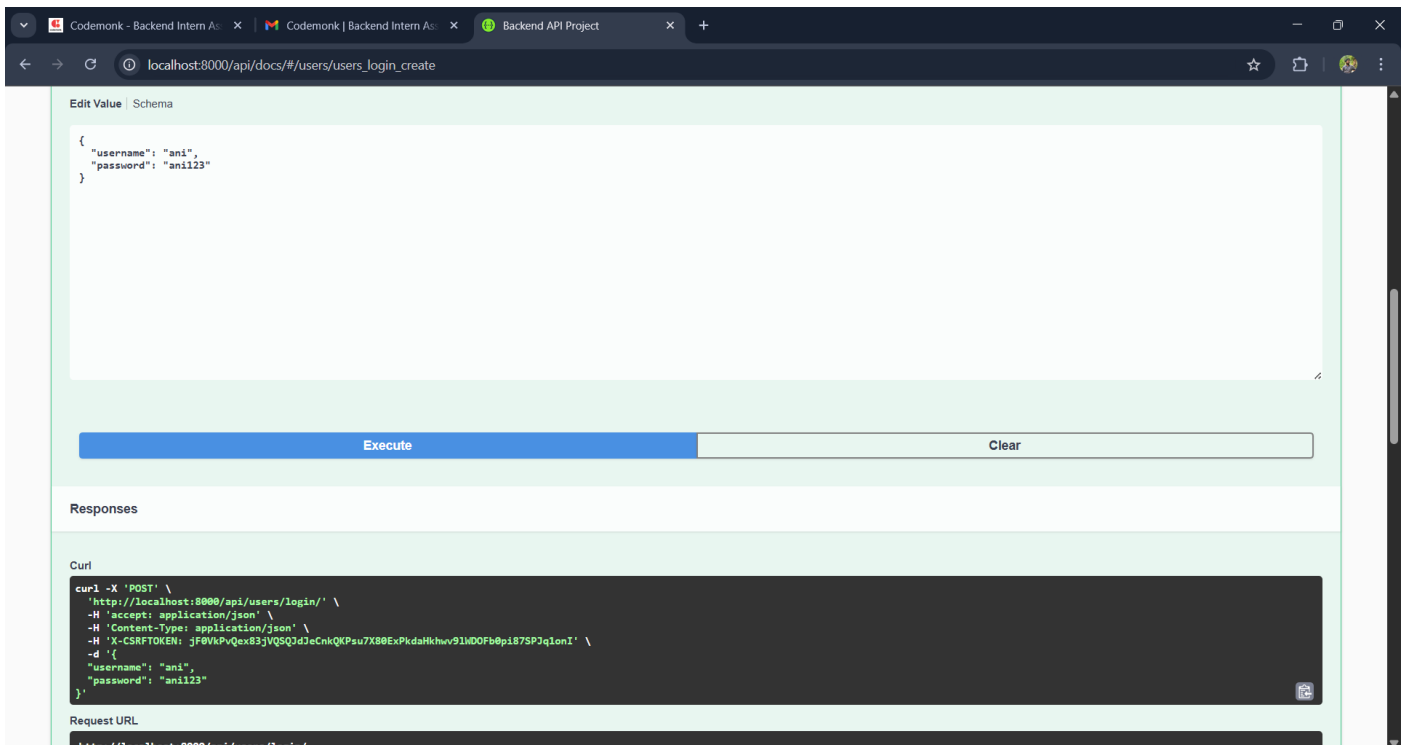
</api/users/login/>



The image shows the Swagger UI for the POST endpoint `/api/users/login/`. The interface is in a web browser with the URL `localhost:8000/api/docs/#/users/users_login_create`. The endpoint description states: "Takes a set of user credentials and returns an access and refresh JSON web token pair to prove the authentication of those credentials." Under the "Parameters" section, it says "No parameters". The "Request body" section is marked as "required" and has a dropdown menu set to "application/json". An "Example Value" is provided in a dark box:

```
{  "username": "string",  "password": "string"}
```

 The "Responses" section shows a table with one entry: status code 200, description "Media type", and a dropdown menu set to "application/json". The "Links" column for this response is "No links". The bottom of the image shows a Windows taskbar with various application icons and a system clock showing 22:22 on 13-08-2025.



The image shows the Swagger UI after clicking the "Execute" button. The "Edit Value" section is active, showing the example request body:

```
{  "username": "ani",  "password": "ani123"}
```

 Below this, there are "Execute" and "Clear" buttons. The "Responses" section is expanded, showing a "Curl" command that can be copied:

```
curl -X 'POST' \  'http://localhost:8000/api/users/login/' \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -H 'X-CSRFToken: jF0vkPvQex83jVQSGQdJecnkQKPsu7X88ExPkdaHkhwv9lMD0Fb0p187SPJqlonI' \  -d '{  "username": "ani",  "password": "ani123"  }'
```

 Below the curl command, the "Request URL" is displayed as `http://localhost:8000/api/users/login/`. The bottom of the image shows the same Windows taskbar as the first image.

Browser tabs: Codemunk - Backend Intern A..., Codemunk | Backend Intern A..., Backend API Project

URL: localhost:8000/api/docs/#/users/users_login_create

Endpoint: http://localhost:8000/api/users/login/

Server response

Code	Details
200	Response body <pre>{ "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2t1b190eXB1IjoicmVmcVzaCiIsImV4cCI6MTc1NTE5NDE2SwiaWF0IjoxNzU1MTA3MzYxLjQ6Gk10IjMTJjNzFjYzFjMGI8OWRjOG%MG1wODYxODEyODU5NiIsInVzX3JfajVQI0iixIn0.IZf-04wcpP7GMIQV8_NE7SoN0zgxpIU3eeOuV9b-8dQ", "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2t1b190eXB1IjoicmVmcVzaCiIsImV4cCI6MTc1NTE5NDE2SwiaWF0IjoxNzU1MTA3MzYxLjQ6Gk10IjMTJjNzFjYzFjMGI8OWRjOG%MG1wODYxODEyODU5NiIsInVzX3JfajVQI0iixIn0.TiZM09CFHQxun81sg7qQcZsf3ALCZeyC4Ev63LKyto" }</pre> Response headers <pre>allow: POST,OPTIONS content-length: 489 content-type: application/json cross-origin-opener-policy: same-origin date: Wed, 13 Aug 2025 17:56:02 GMT referrer-policy: same-origin server: MSGIServer/0.2 CPython/3.9.23 vary: Accept x-content-type-options: nosniff x-frame-options: DENY</pre>

Responses

Code	Description	Links
200	Media type application/json Controls Accept header. Example Value Schema <pre>{ "access": "string", "refresh": "string" }</pre>	No links

II. Register into app

POST

[/api/users/register/](#)

Browser tabs: Codemunk - Backend Intern A..., Codemunk | Backend Intern A..., Backend API Project

URL: localhost:8000/api/docs/#/users/users_register_create

Endpoint: POST /api/users/register/

Parameters

No parameters

Request body *required*

Example Value | Schema

```
{
  "username": "gg_3C_IVVaxJvn9TyVA0sIbCnZTP-Qwh9V6KCVdn18p19cHioV.t",
  "email": "user@example.com",
  "password": "string"
}
```

Responses

Code	Description	Links
201	Media type application/json Controls Accept header. Example Value Schema <pre>{ "username": "V_vId3ZU6eU7AEGUgJNfK7Ij", "email": "user@example.com" }</pre>	No links

Codemunk - Backend Intern A...

Codemunk | Backend Intern A...

Backend API Project

+

localhost:8000/api/docs/#/users/users_token_refresh_create

POST /api/users/register/

Parameters

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "username": "james",
  "email": "james@gmail.com",
  "password": "james123"
}
```

Execute

Clear

Responses

Codemunk - Backend Intern A...

Codemunk | Backend Intern A...

Backend API Project

+

localhost:8000/api/docs/#/users/users_token_refresh_create

Curl

```
curl -X 'POST' \
  'http://localhost:8000/api/users/register/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -H 'X-CSRFToken: jF0vKpVQex83jVQ5Q3dJcCnkQKPsu7X80ExPkdaHkhw91M00Fb0pi87SPJqlenI' \
  -d '{
    "username": "james",
    "email": "james@gmail.com",
    "password": "james123"
  }'
```

Request URL

http://localhost:8000/api/users/register/

Server response

Code

Details

201

Response body

```
{
  "username": "james",
  "email": "james@gmail.com"
}
```

Download

Response headers

```
allow: POST,OPTIONS
content-length: 46
content-type: application/json
cross-origin-opener-policy: same-origin
date: Wed, 13 Aug 2025 18:07:30 GMT
referrer-policy: same-origin
server: WSGIServer/0.2 CPython/3.9.23
vary: Accept
x-content-type-options: nosniff
x-frame-options: DENY
```

Responses

Code

Description

Links

201

No links

POST

</api/users/token/refresh/>

POST /api/users/token/refresh/

Takes a refresh type JSON web token and returns an access type JSON web token if the refresh token is valid.

Parameters

No parameters

Request body *required*

application/json

Example Value | Schema

```
{
  "refresh": "string"
}
```

Responses

Code	Description	Links
200	Media type application/json Controls Accept header. Example Value Schema	No links

4. Schemas

Paragraph, ParagraphInput, TokenObtainPair

Schemas

Paragraph {

- id* integer (readOnly: true)
- text* string
- created_at* string(\$date-time) (readOnly: true)

}

ParagraphInput {

- text* string (Multiple paragraphs separated by two newlines. Example: 'Para 1\n\nPara 2')
- title* string

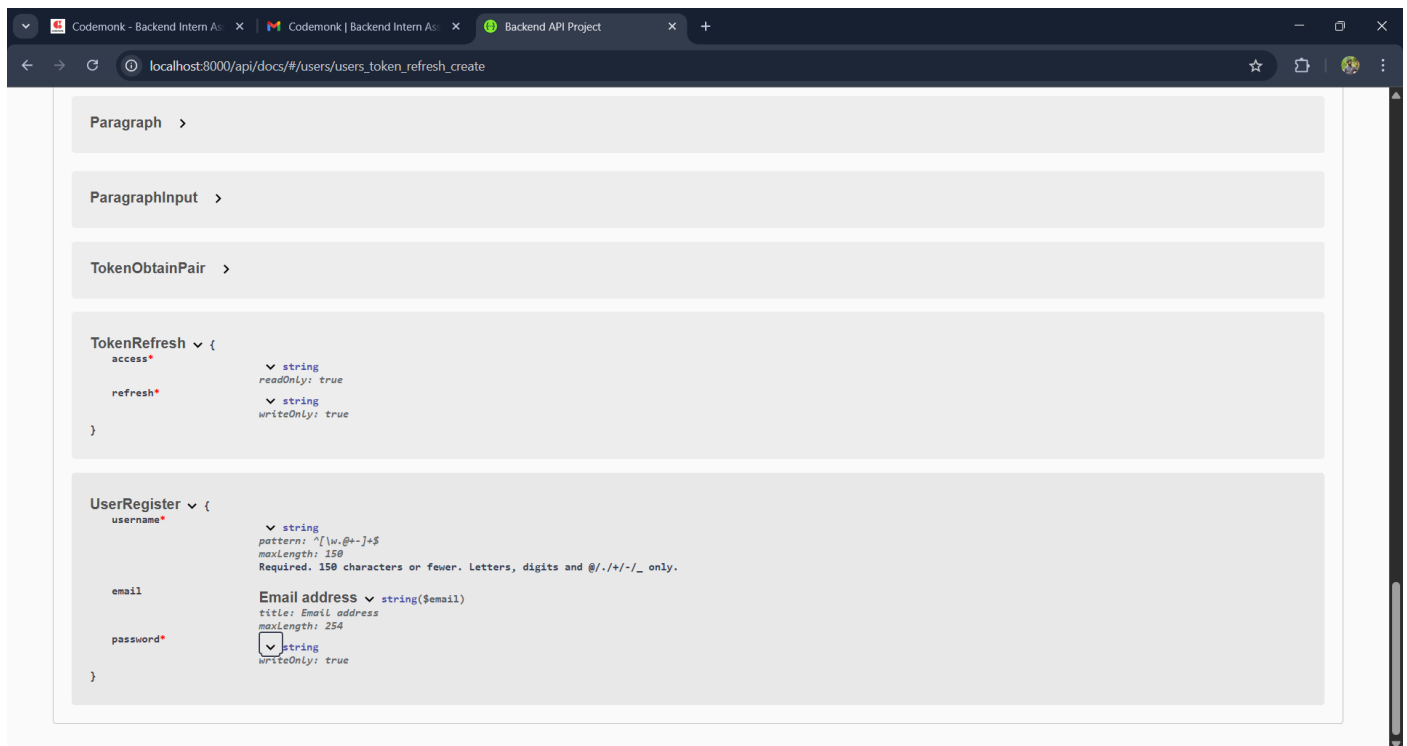
}

TokenObtainPair {

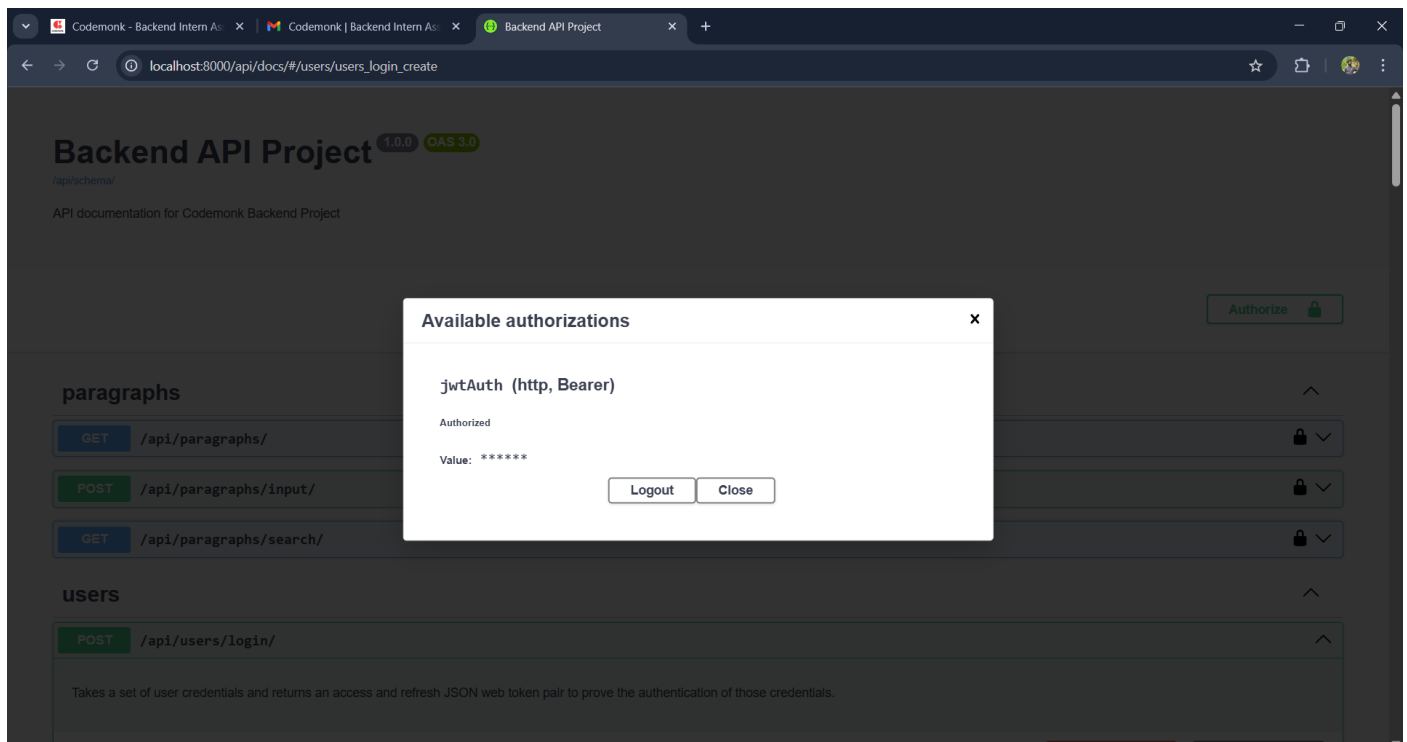
- username* string (writeOnly: true)
- password* string (writeOnly: true)
- access* string (readOnly: true)
- refresh* string (readOnly: true)

}

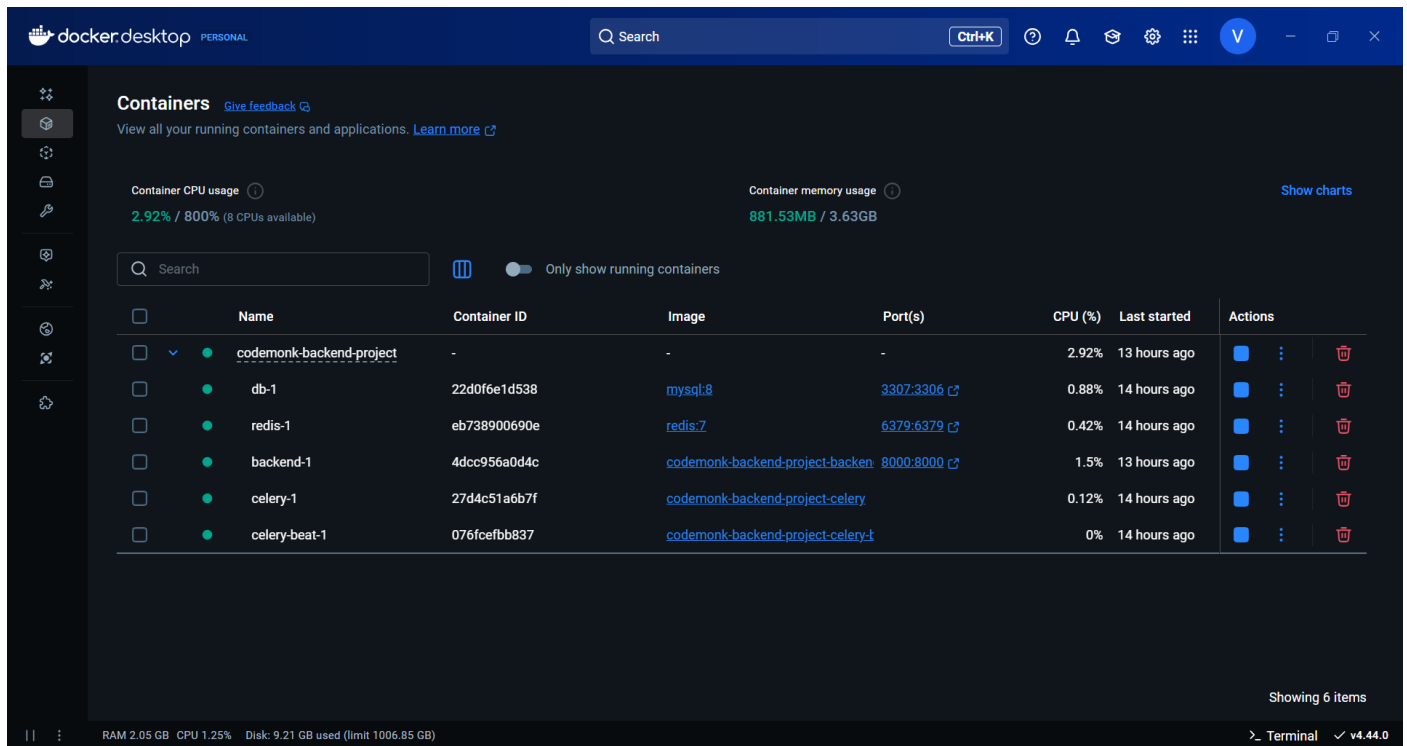
TokenRefresh, UserRegister



5. Authorize



6. Docker Compose



7. Source Code

Backend Module:

- codemonk-backend-project\backend\settings.py

```
8. """
9. Django settings for backend project.
10.
11. Generated by 'django-admin startproject' using Django 4.2.23.
12.
13. For more information on this file, see
14. https://docs.djangoproject.com/en/4.2/topics/settings/
15.
16. For the full list of settings and their values, see
17. https://docs.djangoproject.com/en/4.2/ref/settings/
18. """
19.
20. from pathlib import Path
21. import os # used for environment variables and docker check
22.
23. # build paths inside the project (BASE_DIR points to the root folder of the project)
24. BASE_DIR = Path(__file__).resolve().parent.parent
25.
26. # development settings
27.
28. # secret key for django - keep this secret in production
29. SECRET_KEY = 'django-insecure-u^q^53mr1i4+(1oca4q2-4)8#61t@k!z@jriwwoq3qbtp!29te'
30.
```



```
31.# debug mode - should be False in production
32.DEBUG = True
33.
34.# list of allowed hostnames/domain names for the app
35.ALLOWED_HOSTS = []
36.
37.# application definition
38.
39.# list of installed apps: includes django defaults, third-party apps, and our own apps
40.INSTALLED_APPS = [
41.    # django default apps
42.    'django.contrib.admin',
43.    'django.contrib.auth',
44.    'django.contrib.contenttypes',
45.    'django.contrib.sessions',
46.    'django.contrib.messages',
47.    'django.contrib.staticfiles',
48.
49.    # third-party apps
50.    'rest_framework',          # django rest framework for api building
51.    'rest_framework_simplejwt', # jwt authentication for securing apis
52.    'drf_spectacular',         # api documentation generator (openapi/swagger)
53.
54.    # custom apps
55.    'users',                   # handles user registration, login, and profiles
56.    'paragraphs',              # manages paragraph storage and search features
57.]
58.
59.# middleware: processes requests and responses
60.MIDDLEWARE = [
61.    'django.middleware.security.SecurityMiddleware',          # security headers and
        features
62.    'django.contrib.sessions.middleware.SessionMiddleware',   # session management
63.    'django.middleware.common.CommonMiddleware',              # common
        request/response handling
64.    'django.middleware.csrf.CsrfViewMiddleware',              # csrf protection for
        forms
65.    'django.contrib.auth.middleware.AuthenticationMiddleware', # associates users with
        requests
66.    'django.contrib.messages.middleware.MessageMiddleware',    # messaging framework
67.    'django.middleware.clickjacking.XFrameOptionsMiddleware',  # prevents clickjacking
        attacks
68.]
69.
70.# root urls module
71.ROOT_URLCONF = 'backend.urls'
72.
73.# template engine configuration
74.TEMPLATES = [
75.    {
76.        'BACKEND': 'django.template.backends.django.DjangoTemplates', # template
        engine
77.        'DIRS': [],            # extra template directories (none for now)
78.        'APP_DIRS': True,      # look for templates in each app's templates folder
```

```

79.         'OPTIONS': {
80.             'context_processors': [
81.                 'django.template.context_processors.debug',          # adds debug info to
templates
82.                 'django.template.context_processors.request',      # adds request object
to templates
83.                 'django.contrib.auth.context_processors.auth',     # adds user/auth info
to templates
84.                 'django.contrib.messages.context_processors.messages', # adds messages
to templates
85.             ],
86.         },
87.     },
88.]
89.
90.# wsgi entry point - used by servers to run the app
91.WSGI_APPLICATION = 'backend.wsgi.application'
92.
93.# database
94.# check if the app is running inside docker
95.IN_DOCKER = os.path.exists('/.dockerenv') # used to switch host/port for docker
96.
97.DATABASES = {
98.    'default': {
99.        'ENGINE': 'django.db.backends.mysql',          # database backend
100.        'NAME': os.getenv('DB_NAME', 'codemonk'),      # db name (env or
default)
101.        'USER': os.getenv('DB_USER', 'codemonk_user'), # db username
102.        'PASSWORD': os.getenv('DB_PASSWORD', 'anil209'), # db password
103.        'HOST': 'db' if IN_DOCKER else '127.0.0.1',    # db host (docker vs
local)
104.        'PORT': '3306' if IN_DOCKER else '3307',      # db port (docker vs
local)
105.    }
106. }
107.
108. # password validation rules
109. AUTH_PASSWORD_VALIDATORS = [
110.     {
111.         'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidator', # no
passwords too similar to user info
112.     },
113.     {
114.         'NAME':
'django.contrib.auth.password_validation.MinimumLengthValidator',          # enforce
minimum length
115.     },
116.     {
117.         'NAME':
'django.contrib.auth.password_validation.CommonPasswordValidator',        # block
common passwords
118.     },
119.     {

```

```

120.         'NAME':
121.         'django.contrib.auth.password_validation.NumericPasswordValidator',      # block
122.         all-numeric passwords
123.     },
124. ]
125.
126. # celery configuration - used for background task processing
127. CELERY_BROKER_URL = os.environ.get('CELERY_BROKER_URL',
128.     'redis://redis:6379/0')      # broker (task queue)
129. CELERY_RESULT_BACKEND = os.environ.get('CELERY_RESULT_BACKEND',
130.     'redis://redis:6379/0') # backend (stores results)
131.
132. # rest framework settings
133. REST_FRAMEWORK = {
134.     'DEFAULT_AUTHENTICATION_CLASSES': (
135.         'rest_framework_simplejwt.authentication.JWTAuthentication', # use jwt auth
136.     ),
137.     'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',      # auto-
138.     generate schema for swagger docs
139. }
140.
141. # spectacular (swagger/openapi) documentation settings
142. SPECTACULAR_SETTINGS = {
143.     'TITLE': 'Backend API Project',      # api doc title
144.     'DESCRIPTION': 'API documentation for Codemonk Backend Project', # api doc
145.     description
146.     'VERSION': '1.0.0',      # version number
147.     'SERVE_INCLUDE_SCHEMA': False,      # don't include schema in
148.     served docs
149. }
150.
151. # internationalization
152. LANGUAGE_CODE = 'en-us'      # language setting
153. TIME_ZONE = 'UTC'      # time zone setting
154. USE_I18N = True      # enable django's internationalization system
155. USE_TZ = True      # enable timezone-aware datetimes
156.
157. # static files (css, js, images)
158. STATIC_URL = 'static/'      # url path for serving static files
159.
160. # default primary key type for models
161. DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
162.

```

- **codemonk-backend-project\backend\urls.py**

```

• """
• URL configuration for backend project.
•
• The `urlpatterns` list routes URLs to views. For more information please see:
•     https://docs.djangoproject.com/en/4.2/topics/http/urls/
• Examples:

```

- Function views
 - 1. Add an import: `from my_app import views`
 - 2. Add a URL to urlpatterns: `path('', views.home, name='home')`
- Class-based views
 - 1. Add an import: `from other_app.views import Home`
 - 2. Add a URL to urlpatterns: `path('', Home.as_view(), name='home')`
- Including another URLconf
 - 1. Import the include() function: `from django.urls import include, path`
 - 2. Add a URL to urlpatterns: `path('blog/', include('blog.urls'))`
- """
- `from django.contrib import admin`
- `from django.urls import path, include`
- `from django.shortcuts import redirect`
-
- `from drf_spectacular.views import SpectacularAPIView, SpectacularSwaggerView`
- `from drf_spectacular.utils import extend_schema`
- `from rest_framework.views import APIView`
- `from rest_framework.response import Response`
- `from rest_framework import status`
-
- `# home view to redirect root path to /api/docs/`
- `def home(request):`
- `return redirect('/api/docs/')`
-
- `urlpatterns = [`
- `path('', home), # home view`
- `path('admin/', admin.site.urls), # django admin interface`
- `path('api/users/', include('users.urls')), # user management routes`
- `path('api/paragraphs/', include('paragraphs.urls')), # paragraph features routes`
- `path('api/schema/', SpectacularAPIView.as_view(), name='schema'), # openapi schema`
- `path('api/docs/', SpectacularSwaggerView.as_view(url_name='schema'), name='swagger-`
- `ui'), # swagger ui`
- `]`
-

- **codemonk-backend-project\backend\cerely.py**

- """
- celery configuration for the django backend project.
-
- - sets default django settings module
- - creates celery app named 'backend'
- - loads settings with 'celery_' namespace
- - auto-discovers tasks from installed apps
- """
-
- `import os`
- `from celery import Celery`
-
- `os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'backend.settings')`
- `app = Celery('backend')`

- `app.config_from_object('django.conf:settings', namespace='CELERY')`
- `app.autodiscover_tasks()`
-

Paragraphs Module:

- `codemonk-backend-project\paragraphs\apps.py`

```

• from django.apps import AppConfig # import base app configuration class
•
• class ParagraphsConfig(AppConfig): # configuration for paragraphs app
•     default_auto_field = 'django.db.models.BigAutoField' # default primary key type
•     name = 'paragraphs' # app name
•

```

- `codemonk-backend-project\paragraphs\models.py`

```

• from django.db import models
• from django.contrib.auth.models import User
•
•
• """
• represents a paragraph submitted by a specific user.
• stores the paragraph text, the user who submitted it,
• and the date/time it was created.
• """
• class Paragraph(models.Model):
•     user = models.ForeignKey(User, on_delete=models.CASCADE, related_name='paragraphs')
•     text = models.TextField()
•     created_at = models.DateTimeField(auto_now_add=True)
•
•     def __str__(self):
•         return f"Paragraph {self.id} by {self.user.username}"
•
•
• """
• tracks how many times a specific word appears in all
• paragraphs of a particular user. ensures each (user, word)
• pair is unique to avoid duplicate counts.
• """
• class WordFrequency(models.Model):
•     user = models.ForeignKey(User, on_delete=models.CASCADE,
• related_name='word_frequencies')
•     word = models.CharField(max_length=100)
•     frequency = models.PositiveIntegerField(default=0)
•
•     class Meta:
•         unique_together = ('user', 'word')
•
•     def __str__(self):
•         return f"{self.word}: {self.frequency} (User: {self.user.username})"
•

```

- **codemonk-backend-project\paragraphs\serializers.py**

```
• from rest_framework import serializers
• from .models import Paragraph
•
• """
• serializes the paragraph model for api responses.
• includes paragraph id, text content, and creation date.
• """
• class ParagraphSerializer(serializers.ModelSerializer):
•     class Meta:
•         model = Paragraph
•         fields = ['id', 'text', 'created_at']
•
•
• """
• handles the input for searching a specific word
• across stored paragraphs.
• """
• class WordSearchSerializer(serializers.Serializer):
•     word = serializers.CharField()
•
•
• """
• handles input for adding multiple paragraphs at once.
• paragraphs should be separated by two newlines, and an
• optional title can be provided.
• """
• class ParagraphInputSerializer(serializers.Serializer):
•     text = serializers.CharField(
•         help_text="Multiple paragraphs separated by two newlines. Example: 'Para
1\\n\\nPara 2'"
•     )
•     title = serializers.CharField(required=False)
•
•
• # serializer for search results with count
• class ParagraphSearchResultSerializer(serializers.ModelSerializer):
•     count = serializers.IntegerField() # number of times the word appears
•
•     class Meta:
•         model = Paragraph
•         fields = ['id', 'text', 'count']
•
•
```

- **codemonk-backend-project\paragraphs\urls.py**

```
• from django.urls import path
• from .views import ParagraphInputView, WordSearchView, ParagraphListView
•
• # url patterns for paragraph-related endpoints
• urlpatterns = [
•     path('input/', ParagraphInputView.as_view(), name='paragraph_input'), # endpoint
for adding paragraphs
•
```

- `path('search/', WordSearchView.as_view(), name='word_search'), # endpoint for searching word occurrences`
- `path('', ParagraphListView.as_view(), name='paragraph_list'), # endpoint for listing all paragraphs`
- `]`
-

- **codemonk-backend-project\paragraphs\views.py**

- `from rest_framework import generics, permissions`
- `from rest_framework.views import APIView`
- `from rest_framework.response import Response`
- `from rest_framework import status`
- `from paragraphs.serializers import ParagraphInputSerializer, ParagraphSerializer, WordSearchSerializer`
- `from paragraphs.models import Paragraph, WordFrequency`
- `from django.db.models import F`
- `from django.contrib.auth.models import User`
- `from rest_framework.permissions import IsAuthenticated`
- `from django.db.models import Count`
- `from drf_spectacular.utils import extend_schema`
- `from rest_framework.generics import ListAPIView`
-
- `"""`
- `ParagraphInputView:`
- `- Handles POST requests to input multiple paragraphs for the authenticated user.`
- `- Splits input text into separate paragraphs by two newlines.`
- `- Creates Paragraph objects for each paragraph.`
- `- Updates or creates WordFrequency entries for each word in the paragraphs.`
- `- Returns serialized data of created paragraphs.`
- `"""`
- `class ParagraphInputView(APIView):`
- `permission_classes = [IsAuthenticated]`
-
- `@extend_schema(`
- `request=ParagraphInputSerializer,`
- `responses={201: ParagraphSerializer(many=True)},`
- `description="Input multiple paragraphs separated by two newlines."`
- `)`
- `def post(self, request):`
- `serializer = ParagraphInputSerializer(data=request.data)`
- `if serializer.is_valid():`
- `paragraphs_text = serializer.validated_data['text']`
- `paragraphs = [p.strip() for p in paragraphs_text.split('\n\n') if`
- `p.strip()]`
- `created = []`
- `for para in paragraphs:`
- `p = Paragraph.objects.create(user=request.user, text=para)`
- `created.append(p)`
- `# token and update the words`
- `words = para.split()`
- `for word in words:`

```

    freq_obj, _ =
WordFrequency.objects.get_or_create(user=request.user, word=word)
    freq_obj.frequency = F('frequency') + 1
    freq_obj.save()
    return Response({'created': ParagraphSerializer(created, many=True).data},
status=status.HTTP_201_CREATED)
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

"""
WordSearchView:
- Handles GET requests to search for a specific word in the authenticated user's
paragraphs.
- Counts occurrences of the given word in each paragraph.
- Returns the top 10 paragraphs containing the word, sorted by frequency in descending
order.
"""
import re

class WordSearchView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request):
        word = request.query_params.get('word')
        if not word:
            return Response({'error': 'word parameter is required'}, status=400)
        user_paragraphs = Paragraph.objects.filter(user=request.user)
        para_counts = []
        pattern = re.compile(r'\b{}\b'.format(re.escape(word)), re.IGNORECASE)
        for para in user_paragraphs:
            count = len(pattern.findall(para.text))
            if count > 0:
                para_counts.append({'paragraph': para, 'count': count})
        top = sorted(para_counts, key=lambda x: x['count'], reverse=True)[:10]
        return Response({'results': [
            {'id': x['paragraph'].id, 'text': x['paragraph'].text, 'count': x['count']}
for x in top
        ]})

"""
ParagraphListView:
- Provides a list of all paragraphs created by the authenticated user.
- Uses ParagraphSerializer for serialization.
"""
class ParagraphListView(ListAPIView):
    serializer_class = ParagraphSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        return Paragraph.objects.filter(user=self.request.user)

```


Users Module:

- codemonk-backend-project\users\apps.py

```
• from django.apps import AppConfig # import base app configuration class
•
• class UsersConfig(AppConfig): # configuration for users app
•     default_auto_field = 'django.db.models.BigAutoField' # default primary key type
•     name = 'users' # app name
•
```

- codemonk-backend-project\users\serializers.py

```
• from django.contrib.auth.models import User
• from rest_framework import serializers
•
• """
• UserRegisterSerializer:
• - serializer for user registration.
• - accepts username, email, and password fields.
• - ensures password is write-only (not returned in responses).
• - creates a new User instance using Django's create_user method for proper password
  hashing.
• """
• class UserRegisterSerializer(serializers.ModelSerializer):
•     password = serializers.CharField(write_only=True)
•
•     class Meta:
•         model = User
•         fields = ('username', 'email', 'password')
•
•     def create(self, validated_data):
•         user = User.objects.create_user(
•             username=validated_data['username'],
•             email=validated_data['email'],
•             password=validated_data['password']
•         )
•         return user
•
```

- Codemonk-backend-project\users\urls.py

```
• from django.urls import path
• from .views import RegisterView
• from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView
•
• # URL patterns for authentication-related endpoints
• urlpatterns = [
•     path('register/', RegisterView.as_view(), name='register'), # user registration
  endpoint
•     path('login/', TokenObtainPairView.as_view(), name='login'), # JWT token obtain
  endpoint
•     path('token/refresh/', TokenRefreshView.as_view(), name='token_refresh'), # JWT
  token refresh endpoint
•
```

-]
-

- **codemonk-backend-project\users\views.py**

```
• from rest_framework import generics
• from django.contrib.auth.models import User
• from .serializers import UserRegisterSerializer
•
• """
• - view for user registration.
• - uses Django REST Framework's CreateAPIView to handle POST requests for creating new
  users.
• """
• class RegisterView(generics.CreateAPIView):
•     queryset = User.objects.all() # retrieves all user records from the database
•     serializer_class = UserRegisterSerializer # serializer used for validating and
  creating user instances
•
```

Docker Files:

- **codemonk-backend-project\docker-compose.yml**

```
• version: '3.8'
•
• services:
•     # mysql 8 database service
•     # stores persistent project data (paragraphs, users, word frequencies, etc.)
•     # port 3307 is used on the host to avoid conflicts with any local mysql installation
•     db:
•         image: mysql:8
•         restart: always
•         environment:
•             MYSQL_DATABASE: codemonk           # database name
•             MYSQL_USER: codemonk_user          # application database user
•             MYSQL_PASSWORD: anil209            # password for mysql_user
•             MYSQL_ROOT_PASSWORD: anil209       # root user password
•         ports:
•             - '3307:3306'                      # host port : container port mapping
•         volumes:
•             - db_data:/var/lib/mysql          # persistent storage for mysql data
•
•     # redis service
•     # used as a message broker and result backend for celery task queue processing
•     redis:
•         image: redis:7
•         restart: always
•         ports:
•             - '6379:6379'                      # default redis port
•
•     # django backend service
•     # runs the web application with database migrations applied on startup
```

```

• backend:
•   build: .
•   command: sh -c "python manage.py migrate && python manage.py runserver
0.0.0.0:8000"
•   volumes:
•     - ./code # mount local code directory into container
•   ports:
•     - '8000:8000' # expose django development server
•   depends_on:
•     - db
•     - redis
•   environment:
•     - DJANGO_SETTINGS_MODULE=backend.settings
•     - CELERY_BROKER_URL=redis://redis:6379/0
•     - CELERY_RESULT_BACKEND=redis://redis:6379/0
•     - MYSQL_DATABASE=codemonk
•     - MYSQL_USER=codemonk_user
•     - MYSQL_PASSWORD=anil209
•     - MYSQL_ROOT_PASSWORD=anil209
•
• # celery worker service
• # processes background tasks asynchronously using redis as the broker
• celery:
•   build: .
•   command: celery -A backend worker -l info
•   volumes:
•     - ./code
•   depends_on:
•     - backend
•     - redis
•   environment:
•     - DJANGO_SETTINGS_MODULE=backend.settings
•     - CELERY_BROKER_URL=redis://redis:6379/0
•     - CELERY_RESULT_BACKEND=redis://redis:6379/0
•
• # celery beat scheduler service
• # manages and schedules periodic tasks for the project
• celery-beat:
•   build: .
•   command: celery -A backend beat -l info
•   volumes:
•     - ./code
•   depends_on:
•     - backend
•     - redis
•   environment:
•     - DJANGO_SETTINGS_MODULE=backend.settings
•     - CELERY_BROKER_URL=redis://redis:6379/0
•     - CELERY_RESULT_BACKEND=redis://redis:6379/0
•
• # volume for persistent database storage
• volumes:
•   db_data:
•

```

- codemonk-backend-project\ **Dockerfile**

```
• # Dockerfile
• FROM python:3.9-slim
•
• ENV PYTHONDONTWRITEBYTECODE 1
• ENV PYTHONUNBUFFERED 1
•
• WORKDIR /code
•
• # installing system dependencies for mysqlclient
• RUN apt-get update \
•     && apt-get install -y gcc default-libmysqlclient-dev pkg-config \
•     && rm -rf /var/lib/apt/lists/*
•
• COPY requirements.txt /code/
• RUN pip install --upgrade pip && pip install -r requirements.txt
•
• COPY . /code/
•
```

Requirements:

- codemonk-backend-project\ **requirements.txt**

```
• Django>=4.2
• mysqlclient
• celery
• redis
• djangorestframework
• djangorestframework-simplejwt
• drf-spectacular
•
```

README:

- codemonk-backend-project\ **README.md**

```
• # Project Setup Instructions
•
• ## Prerequisites
• - Docker & Docker Compose
• - Python 3.9+
• - MySQL 8+
• - Redis 7+
•
• ## Local Development
•
• 1. **Clone the repository**
• 2. **Create and activate a virtual environment:**
•     ```powershell
•     python -m venv venv
•     .\venv\Scripts\activate
•     ```
• 3. **Install dependencies:**
```

```

•   ```powershell
•   pip install -r requirements.txt
•   ```
•
• 4. **Configure environment variables** (optional, see `docker-compose.yml` for
• defaults)
•
• 5. **Run migrations:**
•   ```powershell
•   python manage.py migrate
•   ```
•
• 6. **Run the development server:**
•   ```powershell
•   python manage.py runserver
•   ```
•
• 7. **Start Celery worker (in a new terminal):**
•   ```powershell
•   celery -A backend worker -l info
•   ```
•
• 8. **Start Celery beat (in a new terminal):**
•   ```powershell
•   celery -A backend beat -l info
•   ```
•
•
• ## Dockerized Development
•
•
• 1. **Build and start all services:**
•   ```powershell
•   docker-compose up --build
•   ```
•
•
• ## API Documentation
•
• - Swagger UI: [http://localhost:8000/api/docs/](http://localhost:8000/api/docs/)
•
• - OpenAPI schema:
• [http://localhost:8000/api/schema/](http://localhost:8000/api/schema/)
•
•
• ## User APIs
•
• - `POST /api/users/register/` – Register a new user
•
• - `POST /api/users/login/` – Obtain JWT token
•
•
• ## Paragraph APIs
•
• - `POST /api/paragraphs/input/` – Input multiple paragraphs (separated by two newlines)
•
• - `GET /api/paragraphs/search/?word=example` – Get top 10 paragraphs with max
• occurrences of a word
•
•
• ## Code Documentation
•
• - All modules and classes are documented inline.
•
•
• ---
•
•
• For any issues, please refer to the code comments and API docs.
•

```